

Patrones de diseño: Flyweight y Memento

Programación Orientada a Objetos — documentación de apoyo para los ejemplos en C#.

1. Flyweight (Peso Ligero)

Tipo: Patrón estructural

Problema: Cuando una aplicación necesita crear una cantidad enorme de objetos parecidos (letras de un editor, partículas de un videojuego, celdas de una hoja de cálculo), crear un objeto completo por cada uno consume demasiada memoria.

Idea central: Dividir el estado de un objeto en dos partes: el estado intrínseco (se repite entre objetos y se puede compartir) y el estado extrínseco (es único de cada uso y no se puede compartir). Solo se crea un objeto por cada combinación distinta de estado intrínseco; el estado extrínseco se pasa como parámetro cuando se necesita.

Estado intrínseco vs. extrínseco en el ejemplo

- Intrínseco (vive dentro del objeto Glifo): la letra y la fuente. No cambia una vez creado.
- Extrínseco (se pasa por parámetro): la posición x, y donde se dibuja esa letra. Cambia en cada uso.

Roles y clases del código

Rol	Clase en el código	Responsabilidad
Interfaz del flyweight	IGlifo	Define el contrato Dibujar(x, y). La fábrica y el cliente solo conocen esta interfaz.
Flyweight	Glifo : IGlifo	Guarda el estado intrínseco (letra, fuente) y lo usa en Dibujar().
Fábrica de flyweights	FabricaDeGlifos	Busca un IGlifo ya creado antes de crear uno nuevo; evita duplicados.
Cliente	Program	Pide glifos a la fábrica y les pasa la posición (estado extrínseco) al dibujar.

¿Por qué usar la interfaz IGlifo?

El Dictionary de la fábrica y la variable del cliente están tipados como IGlifo, no como Glifo. Esto significa que si mañana se agrega otro tipo de glifo (por ejemplo, uno con negrita o cursiva), la fábrica y el cliente no necesitan cambiar: solo trabajan con el contrato Dibujar(x, y), sin importar la clase concreta detrás.

Cómo leer el código paso a paso

- Program recorre el texto letra por letra.
- Por cada letra, llama a fabrica.ObtenerGlifo(letra, fuente), que devuelve un IGlifo.
- La fábrica revisa su Dictionary: si ya existe un Glifo para esa combinación, lo devuelve; si no, lo crea y lo guarda.
- Program llama a glifo.Dibujar(x, y) sobre la interfaz, pasando la posición como parámetro.

- Al final se compara el número de letras dibujadas contra el número de objetos Glifo creados: siempre hay menos objetos que letras, porque las repetidas comparten instancia.

Cuándo usarlo

- Se necesitan muchísimas instancias que comparten la mayor parte de sus datos.
- El costo de buscar un flyweight existente es menor que crear uno nuevo.
- Se requiere reducir el uso de memoria sin cambiar la interfaz pública de las clases.

2. Memento (Recuerdo)

Tipo: Patrón de comportamiento

Problema: Se necesita poder deshacer cambios (undo/redo) en un objeto, pero sin exponer sus atributos internos al resto del programa, porque eso rompería el encapsulamiento.

Idea central: El propio objeto (el que sabe su estructura interna) es responsable de crear una 'foto' de su estado. Esa foto se entrega a un tercero que la guarda sin mirar ni modificar su contenido, y que la devuelve cuando se necesite restaurar.

Roles y clases del código

Rol	Clase en el código	Responsabilidad
Interfaz del memento	IMemento	Interfaz vacía (sin métodos ni propiedades). Es lo único que puede ver el Historial.
Memento	EditorMemento : IMemento	Instantánea inmutable del estado (Contenido). Solo el Editor conoce esta clase concreta.
Originator	Editor	El objeto real. Crea (GuardarEstado) y aplica (Restaurar) sus propias instantáneas; es el único que convierte IMemento de vuelta a EditorMemento.
Caretaker	Historial	Guarda objetos IMemento en una pila (Stack) y los entrega en orden inverso, sin poder ver su contenido.
Cliente	Program	Coordina: pide guardar, escribe cambios, pide deshacer.

¿Por qué usar la interfaz IMemento?

IMemento no tiene ni un solo miembro. Su función no es dar comportamiento, sino ocultar información: Historial guarda y devuelve objetos IMemento, pero como la interfaz está vacía, no puede leer ni modificar Contenido aunque quisiera. Solo Editor conoce el tipo concreto EditorMemento y hace el casteo ((EditorMemento)memento) para recuperar el texto. Así se cumple el objetivo del patrón: restaurar el estado sin exponer los detalles internos al resto del programa.

Cómo leer el código paso a paso

- editor.Escribir(texto) modifica el contenido del Editor.

- `historial.Guardar(editor.GuardarEstado())` le pide al Editor una foto de sí mismo (como `IMemento`) y la apila en el Historial.
- Se repite: escribir y, opcionalmente, guardar un nuevo punto de restauración.
- `historial.Deshacer()` saca (`Pop`) el último `IMemento` guardado de la pila.
- `editor.Restaurar(memento)` hace el casteo a `EditorMemento` y reemplaza el contenido actual.
- El cambio 'ERROR123' del ejemplo nunca se guarda en el historial, por eso desaparece apenas se deshace una vez.

Por qué se usa una pila (Stack)

Un Stack devuelve siempre el último elemento agregado (LIFO). Esto encaja naturalmente con undo: el primer 'deshacer' debe regresar al estado más reciente, no al más antiguo.

Cuándo usarlo

- Se necesita historial de cambios para deshacer/rehacer.
- El objeto tiene estructura interna compleja que no debe exponerse al cliente.
- Se quieren tomar snapshots para auditoría, replicación o simulaciones.

Diferencia clave entre ambos patrones

Flyweight es estructural: resuelve cómo organizar objetos para ahorrar memoria compartiendo estado. Memento es de comportamiento: resuelve cómo guardar y restaurar el estado de un objeto en el tiempo, sin romper encapsulamiento. No compiten entre sí; responden preguntas distintas dentro del diseño orientado a objetos.